# Low Level Design (LLD)

Satellite Image Threat Detection - Detailed Module & Class Design

Project: Satellite Image Threat Detection (SITP)

GitHub: github.com/harshal3558/Satellite-Image-Threat-Detection

# [LLD] Low Level Design (LLD)

# Satellite Image Threat Detection (SITP)

## 1. Module Overview

```
src/SITP/
+-- components/
|   +-- data_ingestion.py       # Stage 1: Parse GeoJSON, split images
|   +-- data_transformation.py  # Stage 2: Tile GeoTIFFs -> YOLO chips
|   +-- model_trainer.py        # Stage 3: YOLOv8 fine-tuning
|   \-- model_monitoring.py     # Inference: Tiled prediction + NMS
+-- pipelines/
|   +-- training_pipeline.py    # Orchestrates Stages 1-3
|   \-- prediction_pipeline.py  # Wraps ModelMonitoring for web app
+-- exception.py                # Custom exception with traceback
+-- logger.py                   # Logging setup
\-- utils.py                    # Shared helpers
```

## 2. Class Diagrams

### 2.1 Data Ingestion

```
+-----------------------------------------------------------+
|                   DataIngestionConfig                     |
+-----------------------------------------------------------+
| + raw_annotations_path: str = "artifacts/raw_ann.csv"   |
| + val_fraction: float = 0.2                               |
| + seed: int = 42                                          |
\-----------------------------------------------------------+

+-----------------------------------------------------------+
|                      DataIngestion                        |
+-----------------------------------------------------------+
| - config: DataIngestionConfig                             |
+-----------------------------------------------------------+
| + __init__(config?)                                       |
| + initiate_data_ingestion()                               |
|   -> (ann_df, image_split, image_dir, output_dir)         |
|                                                           |
| [private]                                                 |
| - _load_annotations(label_path) -> pd.DataFrame           |
| - _filter_existing_images(ann_df, image_dir) -> DataFrame|
| - _split_images(ann_df) -> dict[str, str]                 |
\-----------------------------------------------------------+
```

### 2.2 Data Transformation

```
+-------------------------------------------------------------+
|                  DataTransformationConfig                   |
+-------------------------------------------------------------+
| + chip_size: int = 512                                      |
| + stride: int = 364                                         |
| + visibility_threshold: float = 0.4                         |
| + background_keep_prob: float = 0.08                        |
| + use_augmentation: bool = True                             |
| + seed: int = 42                                            |
\-------------------------------------------------------------+

+-------------------------------------------------------------+
|                     DataTransformation                      |
+-------------------------------------------------------------+
| - config: DataTransformationConfig                          |
+-------------------------------------------------------------+
| + __init__(config?)                                         |
| + initiate_data_transformation(ann_df, image_split,         |
|     image_dir, output_dir)                                  |
|   -> (data_yaml_path, class_mapping)                        |
|                                                             |
| [private]                                                   |
| - _build_class_mapping(ann_df) -> dict[int, int]            |
| - _make_transform() -> A.Compose                            |
| - _calculate_visibility(box_coords, chip_polygon) -> float  |
| - _create_yolo_chips(...) -> Counter                        |
| - _write_data_yaml(output_dir, class_mapping) -> Path       |
\-------------------------------------------------------------+
```

## 2.3 Model Trainer

```
+-------------------------------------------------------------+
|                    ModelTrainerConfig                       |
+-------------------------------------------------------------+
| + model_weights: str = "yolov8m.pt"                         |
| + epochs: int = 50                                          |
| + imgsz: int = 512                                          |
| + batch: int = 8                                            |
| + workers: int = 2                                          |
| + optimizer: str = "auto"                                   |
| + seed: int = 42                                            |
| + mosaic: float = 1.0                                       |
| + copy_paste: float = 0.2                                   |
| + degrees: float = 90.0                                     |
| + hsv_h: float = 0.015                                      |
| + hsv_s: float = 0.7                                        |
| + hsv_v: float = 0.4                                        |
| + scale: float = 0.5                                        |
| + translate: float = 0.1                                    |
| + run_name: str = "satellite_detector"                      |
\-------------------------------------------------------------+

+-------------------------------------------------------------+
|                      ModelTrainer                           |
+-------------------------------------------------------------+
| - config: ModelTrainerConfig                                |
+-------------------------------------------------------------+
| + __init__(config?)                                         |
```

```
| + initiate_model_trainer(data_yaml, output_dir)              |
|   -> (YOLO model, best_weights_path)                         |
|                                                              |
| [private]                                                    |
| - _train_model(data_yaml, output_dir) -> YOLO                |
| - _validate_model(model) -> None                             |
\--------------------------------------------------------------+
```

### 2.4 Prediction Pipeline & Model Monitoring

```
+--------------------------------------------------------------+
|                      PredictPipeline                         |
+--------------------------------------------------------------+
| - model_path: Path                                           |
| - tile_size: int = 512                                       |
| - overlap: int = 100                                         |
| - conf: float = 0.25                                         |
| - iou: float = 0.45                                          |
+--------------------------------------------------------------+
| + __init__(model_path?, tile_size?, overlap?, conf?, iou?)   |
| + predict(image_path) -> list[list[float]]                   |
|     returns [[x1, y1, x2, y2, score, class_id], ...]         |
\--------------------------------------------------------------+
            | delegates to
            ?
+--------------------------------------------------------------+
|                     ModelMonitoring                          |
+--------------------------------------------------------------+
| [static methods]                                             |
| + predict_large_image(image_path, model_path,                |
|     tile_size, overlap, conf_threshold, iou_threshold)       |
|   -> list[list[float]]                                       |
|                                                              |
| + run_nms(detections, iou_threshold)                         |
|   -> list[list[float]]                                       |
\--------------------------------------------------------------+
```

### 2.5 Pipeline Orchestrators

```
+--------------------------------------------------------------+
|                     TrainingPipeline                         |
+--------------------------------------------------------------+
| - ingestion_config: DataIngestionConfig                      |
| - transformation_config: DataTransformationConfig            |
| - trainer_config: ModelTrainerConfig                         |
+--------------------------------------------------------------+
| + __init__(ingestion_config?, transformation_config?,        |
|            trainer_config?)                                  |
| + run() -> Path  (returns best.pt path)                      |
\--------------------------------------------------------------+
```

## 3. Utility Functions (`utils.py`)

| Function | Signature | Description |
|---|---|---|

| | | |
|---|---|---|
| seed_everything | (seed: int) -> None | Seeds Python, NumPy, PyTorch |
| print_device_info | () -> None | Logs CUDA/CPU availability |
| get_paths | () -> (image_dir, label_path, output | Auto-detects Kaggle vs local paths |
| prepare_dirs | (output_dir: Path) -> None | Creates images/train, images/val, l |
| normalize_to_uint8 | (chip: np.ndarray) -> np.ndarray | 1-99 percentile clipping to uint8 |
| tile_starts | (length, tile_size, stride) -> list[ | Sliding-window tile positions |
| save_chip | (chip, boxes_yolo, output_dir, spli | Writes JPEG chip + YOLO .txt label |

## 4. Data Structures

### 4.1 Annotation DataFrame (`ann_df`)

| Column | Type | Description |
|---|---|---|
| image_id | str | Filename of the source GeoTIFF (e.g |
| type_id | int | Raw xView class ID |
| x1 | int | Left pixel coordinate (image space) |
| y1 | int | Top pixel coordinate (image space) |
| x2 | int | Right pixel coordinate (image space |
| y2 | int | Bottom pixel coordinate (image spac |

### 4.2 Image Split Dictionary

```
image_split: dict[str, str]
# Example:
{
    "1234.tif": "train",
    "5678.tif": "val",
    ...
}
```

### 4.3 Class Mapping

```
class_mapping: dict[int, int]
# Maps raw xView type_id -> contiguous YOLO class index (0-based)
# Example:
{
    11: 0,   # xView type 11 -> YOLO class 0
    12: 1,   # xView type 12 -> YOLO class 1
    ...
}
```

### 4.4 YOLO Label Format (per chip)

Each .txt label file row:

```
<class_idx> <x_center> <y_center> <width> <height>
```

All values normalised to [0.0, 1.0] relative to chip dimensions (512×512).

```
detections: list[list[float]]
# Each detection: [x1, y1, x2, y2, confidence_score, class_id]
# Coordinates are in global image space (pixels)
```

# 5. Algorithm Details

## 5.1 Sliding-Window Tiling (`tile_starts`)

```
Input: length=5000, tile_size=512, stride=364

Step 1: Generate starts = [0, 364, 728, 1092, ..., 4488]
Step 2: Append final start = 5000 - 512 = 4488 (if not already present)
        -> ensures full image coverage at boundaries

Result: list of pixel offsets covering the full dimension
```

## 5.2 Visibility Threshold Filtering

```
For each annotation box (x1,y1,x2,y2) and chip window:

  obj_box = Shapely box(x1, y1, x2, y2)
  chip_polygon = Shapely box(cx, cy, cx+512, cy+512)

  visibility = intersection.area / obj_box.area

  Include annotation in chip only if visibility >= 0.4
  (at least 40% of the object is visible within the chip)
```

## 5.3 YOLO Coordinate Conversion

```
For a bounding box (x1, y1, x2, y2) in chip-local pixel space:

  nx1 = max(0, x1 - chip_x_offset)
  nx2 = min(chip_size, x2 - chip_x_offset)
  ny1 = max(0, y1 - chip_y_offset)
  ny2 = min(chip_size, y2 - chip_y_offset)

  x_center = ((nx1 + nx2) / 2.0) / chip_size   ? [0, 1]
  y_center = ((ny1 + ny2) / 2.0) / chip_size   ? [0, 1]
  width    = (nx2 - nx1) / chip_size             ? [0, 1]
  height   = (ny2 - ny1) / chip_size             ? [0, 1]
```

## 5.4 Image Normalization (`normalize_to_uint8`)

```
1. Cast to float32
2. Compute p1 = 1st percentile, p99 = 99th percentile
3. Clip: chip = clip(chip, p1, p99)
4. Scale: chip = (chip - p1) * 255 / (p99 - p1)
```

```
    5. Cast to uint8
    Handles multi-band GeoTIFFs with extreme sensor value ranges.
```

### 5.5 Tiled Inference & Global NMS (Inference Phase)

```
For each tile (tx, ty) with overlap:
  1. Read 512×512 chip from GeoTIFF using rasterio.Window
  2. Run YOLOv8 on chip -> local detections [lx1,ly1,lx2,ly2,score,cls]
  3. Translate to global coords:
       gx1 = lx1 + tx
       gy1 = ly1 + ty
       gx2 = lx2 + tx
       gy2 = ly2 + ty
  4. Accumulate all global detections

After all tiles:
  5. Run batched NMS on accumulated detections with iou_threshold
  6. Return final deduplicated detections
```

## 6. Flask Web Application Routes

| Method | Route | Function | Description |
|---|---|---|---|
| GET | / | index() | Renders upload page (index.html) |
| POST | / | index() | Handles upload, runs inference, ret |
| GET | /health | health() | Returns {"status": "ok"} for health |

### POST `/` Request Flow

```
1. Validate file extension (.tif / .tiff only)
2. Save with UUID-prefixed filename -> uploads/{uuid}_{original}.tif
3. Parse conf & iou from form data (defaults: conf=0.25, iou=0.45)
4. Run PredictPipeline.predict(filepath) -> detections
5. Read GeoTIFF as RGB numpy array via rasterio
6. Draw bounding boxes with class labels via OpenCV
7. Downscale visualization to <= 1024px longest side
8. Encode visualization as base64 JPEG string
9. Compute statistics:
   - total detections
   - average confidence
   - per-class detection counts + percentages
   - image metadata (width, height, bands, driver, file size)
10. Render index.html with all results
11. Delete uploaded file (finally block)
```

## 7. Configuration Summary

| Parameter | Default | Where Used |
|---|---|---|
| val_fraction | 0.2 | DataIngestion - 20% images for vali |

| | | |
|---|---|---|
| chip_size | 512 | DataTransformation, ModelTrainer, P |
| stride | 364 | DataTransformation - sliding window |
| visibility_threshold | 0.4 | DataTransformation - min object vis |
| background_keep_prob | 0.08 | DataTransformation - 8% chance to k |
| epochs | 50 | ModelTrainer |
| batch | 8 | ModelTrainer |
| imgsz | 512 | ModelTrainer - must match chip_size |
| mosaic | 1.0 | ModelTrainer - YOLO mosaic augmenta |
| copy_paste | 0.2 | ModelTrainer - YOLO copy-paste augm |
| overlap (inference) | 100 | PredictPipeline - tile overlap in p |
| conf (inference) | 0.25 | PredictPipeline - confidence thresh |
| iou (inference) | 0.45 | PredictPipeline - NMS IoU threshold |
| seed | 42 | All stages - reproducibility |
| MAX_CONTENT_LENGTH | 500 MB | Flask - max upload size |

## 8. Error Handling

| Component | Exception Type | Handling Strategy |
|---|---|---|
| All components | CustomException | Wraps original exception with sys t |
| DataIngestion | FileNotFoundError | Raised if GeoJSON label file missin |
| DataIngestion | ValueError | Raised if no annotations found or a |
| PredictPipeline | FileNotFoundError | Raised if best.pt not found at any |
| application.py | FileNotFoundError | Returns user-friendly error page |
| application.py | All exceptions | Caught, logged, rendered as error i |

## 9. File I/O Summary

| Stage | Reads | Writes |
|---|---|---|
| Data Ingestion | xView_train.geojson, .tif images | artifacts/raw_annotations.csv |
| Data Transformation | .tif images, ann_df | xview_yolo/images/{train,val}/*.jpg |
| Model Trainer | data.yaml, chip images | xview_yolo/satellite_detector/weigh |
| Prediction | .tif (via upload), best.pt | _(none - results served in memory)_ |
| Logging | - | logs/*.log |

## 10. Augmentation Pipeline (Training Only)

Applied via Albumentations only to train chips that have at least one label:

| Transform | Probability | Effect |
|---|---|---|
| CLAHE | 35% | Contrast Limited Adaptive Histogram |
| RandomRotate90 | 50% | Rotate chip 0°/90°/180°/270° |
| HorizontalFlip | 50% | Mirror horizontally |
| VerticalFlip | 50% | Mirror vertically |
| RandomBrightnessContrast | 35% | Adjust brightness & contrast random |

Bounding boxes are transformed in sync with the image using BboxParams(format="yolo").